SHOWSHELF LAB · FOR THE AFTERNOON TEACHER

Teacher guide: Day 7

Students work through the steps in the Lab on their own. Each step has an explanation, a task, two hints and an answer button, so most of the time they can get unstuck alone. This guide gives you every task, a correct answer and the mistakes students usually make, so you can help quickly.

How to help: first ask them to read the "Not yet" message and the red Console line (if any), then open a hint. Only point at the answer after that. In the Lab, **Help › Teacher view** shows the answer and these notes on every step.

Day 7: Make the page react — The DOM and events

Until now your JavaScript ran once and printed to the Console. Today the page **reacts**. You build the show cards with JavaScript, filter them while someone types, and add a watchlist called **My List** that is still there after a reload.

You work in one file, **app.js**. The page itself, the styles and two small files are ready. Open the read-only tabs above the code box to look at them:

- **data.js** holds `shows`, the 12 shows you know.
- **helpers.js** has functions you already know, like `ratingLabel`, `yearsLabel` and `searchByName`.

Your code grows from step to step: each step starts with your code from the step before.

Step 1. Find an element ~10 min

Teaches: `document.querySelector("#id")` finds an element; `textContent` changes its text.

TASK

Under the comment `Elements`:

1. Find the grid and keep it in a constant: `const grid = document.querySelector("#grid");`
2. As a test, put some text in it, for example "The shows will appear here."
3. Log the element with `console.log(grid);` and look at the Console under the preview.

COMMON MISTAKES

- Forgetting the `#`: `querySelector("grid")` looks for a `<grid>` tag and gives `null`.
- `queryselector` or `QuerySelector`: capital letters matter.
- Writing `grid.textContent` before the `const grid` line.

A CORRECT ANSWER

```
// ... unchanged ...

// ---------- Elements: the parts of the page we change ----------
const grid = document.querySelector("#grid");
grid.textContent = "The shows will appear here.";
console.log(grid);
```

Step 2. Build an element ~15 min

Teaches: `document.createElement("tag")` makes an element; `parent.append(child)` puts it on the page.

TASK

1. Delete the `grid.textContent = ...` line from step 1. The `console.log` can go too.
2. Under `const grid`, create an `article` element with the class `card`.
3. Create an `h3` whose text is the first show's name: `shows[0].name`.
4. Put the `h3` inside the article, and the article inside the grid.

A small white card with "Friends" appears in the preview.

COMMON MISTAKES

- `card.class = "card"`: the property is called `className`.
- Appending the card to the grid before putting the title in it (it works, but it's easier to read in the other order).
- Forgetting `grid.append(card)`, so the card never appears.

A CORRECT ANSWER

```
// ... unchanged ...
// ---------- Elements: the parts of the page we change ----------
const grid = document.querySelector("#grid");

// ---------- A first card ----------
const card = document.createElement("article");
card.className = "card";

const title = document.createElement("h3");
title.textContent = shows[0].name;

card.append(title);
grid.append(card);
```

Step 3. A function that builds a card ~20 min

Teaches: A function can build an element from an object and `return` it.

TASK

Fill in the three TODO lines inside `createCard`, using the helpers from **helpers.js**:

1. The badge shows the first genre: `show.genres[0]`.
2. The rating line reads like `★ 8.8 · Must watch`: the rating, then the label from `ratingLabel(show.rating)`. Copy the `★` and `·` from the comment, or use `*` and `-`.
3. The meta line shows the years: `yearsLabel(show.premiered, show.ended)`.

COMMON MISTAKES

- `badge.textContent = show.genres;` shows every genre with commas. Use `[0]` for the first one.
- Missing `+` between the pieces of the rating text, which gives a syntax error.
- Deleting `return card;`, so `createCard` gives back `undefined`.

NOTE

The code box starts fresh in this step: their step 2 lines were moved into the function. If a student asks where their code went, show them the `card` and `title` lines at the top of `createCard`.

A CORRECT ANSWER

```
// app.js: makes the ShowShelf page react (Day 7)
// Loaded before this file (open the read-only tabs to see them):
//   data.js    -> shows: the 12 shows
//   helpers.js -> ratingLabel, yearsLabel, searchByName, getByGenre, allGenres

// ---------- Elements: the parts of the page we change ----------
const grid = document.querySelector("#grid");

// ---------- One card for one show ----------
// createCard(show) builds a card and gives it back. It doesn't put it on the page.
function createCard(show) {
  const card = document.createElement("article");
  card.className = "card";

  const poster = document.createElement("img");
  poster.className = "poster";
  poster.src = show.image ?? "poster.svg";   // no picture (null)? use poster.svg
  poster.alt = "";                           // the name is under the poster, so no alt text

  const body = document.createElement("div");
  body.className = "card-body";

  const badge = document.createElement("span");
  badge.className = "badge";
  badge.textContent = show.genres[0];

  const title = document.createElement("h3");
  title.textContent = show.name;

  const rating = document.createElement("p");
  rating.className = "rating";
  rating.textContent = "★ " + show.rating + " · " + ratingLabel(show.rating);

  const meta = document.createElement("p");
  meta.className = "meta";
  meta.textContent = yearsLabel(show.premiered, show.ended);
```

```
  body.append(badge, title, rating, meta);
  card.append(poster, body);
  return card;
}

// Try it with the first show
grid.append(createCard(shows[0]));
```

Step 4. A card for every show ~10 min

Teaches: A loop plus a function: one element for every item in an array.

TASK

Replace the last line, `grid.append(createCard(shows[0]));`, with a `for...of` loop that adds a card for **every** show in `shows`.

COMMON MISTAKES

- Keeping the old line, so Friends appears twice.
- `createCard(shows)` (the whole array) instead of `createCard(show)`.

A CORRECT ANSWER

```
// ... unchanged ...
}

// One card for every show
for (const show of shows) {
  grid.append(createCard(show));
}
```

Step 5. Draw any list of shows ~15 min

Teaches: `renderGrid(list)` empties the grid with `replaceChildren()` and draws a card for each show.

TASK

1. Put your loop inside a new function `renderGrid(list)`. Make `grid.replaceChildren();` its first line, and loop over `list`.
2. Under the function, call it with all the shows: `renderGrid(shows);`

COMMON MISTAKES

- Forgetting `grid.replaceChildren()`, so cards pile up.
- Looping over `shows` inside the function, so the parameter is ignored.

A CORRECT ANSWER

```
// ... unchanged ...
}

// ---------- Draw a list of shows in the grid ----------
function renderGrid(list) {
  grid.replaceChildren();   // remove the old cards first
  for (const show of list) {
    grid.append(createCard(show));
  }
}

renderGrid(shows);
```

Step 6. React to typing ~15 min

Teaches: `element.addEventListener("input", () => { ... })` runs code on every change; `.value` is the text in a box.

TASK

1. At the top, under `const grid`, find the search box: `const searchInput = document.querySelector("#search");`
2. At the bottom, add an `"input"` listener to `searchInput`. Inside it, draw only the matching shows: `renderGrid(searchByName(shows, searchInput.value));`

Then type **office** in the search box in the preview, and delete it again.

COMMON MISTAKES

- `searchInput.value` read once outside the listener: it's empty at that moment and never updates.
- Using the `"change"` event, which waits until you leave the box.
- Calling the function instead of giving it: `addEventListener("input", renderGrid(...))`.

A CORRECT ANSWER

```
// ... unchanged ...
// ---------- Elements: the parts of the page we change ----------
const grid = document.querySelector("#grid");
const searchInput = document.querySelector("#search");

// ---------- One card for one show ----------
// ... unchanged ...

renderGrid(shows);

// ---------- Events: what happens when someone uses the page ----------
// Every time the text in the search box changes, show only the shows that match
searchInput.addEventListener("input", () => {
  renderGrid(searchByName(shows, searchInput.value));
});
```

Step 7. Stop the page reloading ~10 min

Teaches: `event.preventDefault()` stops the browser's default action, like a form reloading the page.

TASK

1. At the top, find the form: `const searchForm = document.querySelector("#search-form");`
2. At the bottom, add a `"submit"` listener to `searchForm` that calls `event.preventDefault();`.

Press Enter in the search box again: no notice this time.

COMMON MISTAKES

- Adding the listener to the button with `"click"`: Enter in the box still submits the form.
- `event.preventDefault` without brackets: nothing happens.
- Forgetting the `event` parameter: `event is not defined`.

A CORRECT ANSWER

```
// ... unchanged ...
const grid = document.querySelector("#grid");
const searchInput = document.querySelector("#search");
const searchForm = document.querySelector("#search-form");

// ---------- One card for one show ----------
// ... unchanged ...
  renderGrid(searchByName(shows, searchInput.value));
});

// Pressing Enter sends ("submits") the form. Stop the browser from reloading the page.
searchForm.addEventListener("submit", (event) => {
  event.preventDefault();
});
```

Step 8. When nothing matches ~10 min

Teaches: `element.hidden = condition;` shows or hides an element.

TASK

1. At the top, find the message: `const emptyMessage = document.querySelector("#empty-message");`
2. At the end of `renderGrid`, add one line that hides the message when the list has shows, and shows it when the list is empty.

Search for zzz, then for dark.

COMMON MISTAKES

- The condition the wrong way round: `list.length === 0` hides the message exactly when it's needed.
- Putting the line outside `renderGrid`, so it only runs once.
- `const emptyMessage` at the bottom of the file: "Cannot access before initialization".

A CORRECT ANSWER

```
// ... unchanged ...
const searchInput = document.querySelector("#search");
const searchForm = document.querySelector("#search-form");
const emptyMessage = document.querySelector("#empty-message");

// ---------- One card for one show ----------
// ... unchanged ...
    grid.append(createCard(show));
  }
  emptyMessage.hidden = list.length > 0;   // show the message only when there are no shows
}

// ... unchanged ...
```

Step 9. Genre buttons and one render() ~30 min

Teaches: Keep what can change in a `state` object; events change the state, then call `render()`.

TASK

1. **TODO 1** in `render()`: if `state.genre` is not `""`, keep only that genre, with `getByGenre(list, state.genre)`.
2. **TODO 2** in the chip's click listener: if `state.genre` is already this `genre`, set it to `""`; otherwise set it to `genre`. Then call `render()`.
3. In your search listener, replace the `renderGrid(...)` line with two lines: `state.search = searchInput.value;` and `render();`.
4. Delete the old line `renderGrid(shows);` under the `renderGrid` function. The `render();` at the bottom draws the page now.

Try it: click **Drama**, then type **the**. Click Drama again to see all genres.

COMMON MISTAKES

- Forgetting `render()` after changing `state.genre`: the state changes but the page doesn't.
- Leaving `renderGrid(searchByName(...))` in the search listener, so typing ignores the chosen genre.
- `if (state.genre = genre)` with one `=`: that sets the value instead of comparing.

NOTE

This step is longer than the others. The key idea is item 4: the search listener stops drawing and only changes `state`. If a student is lost, draw three boxes on paper: event, state, render, with arrows in that order.

A CORRECT ANSWER

```
// ... unchanged ...

// ---------- Events: what happens when someone uses the page ----------
// Every time the text in the search box changes: change the state, then draw again
searchInput.addEventListener("input", () => {
  state.search = searchInput.value;
  render();
});

// ... unchanged ...
  event.preventDefault();
});

// ---------- State: everything on the page that can change ----------
const state = {
  search: "",   // the text in the search box
  genre: "",    // the chosen genre ("" means all genres)
};

// ---------- Draw the page from the state ----------
function render() {
  let list = searchByName(shows, state.search);
  if (state.genre !== "") {
    list = getByGenre(list, state.genre);
  }

  renderGrid(list);
  renderGenres();
}

// One button (a "chip") per genre, in the box with the id "genres". The chosen one gets the class
```

```
"active".
const genresBox = document.querySelector("#genres");
function renderGenres() {
  genresBox.replaceChildren();
  for (const genre of allGenres(shows)) {
    const chip = document.createElement("button");
    chip.className = "chip";
    chip.textContent = genre;
    if (genre === state.genre) {
      chip.classList.add("active");
    }
    chip.addEventListener("click", () => {
      if (state.genre === genre) {
        state.genre = "";      // clicked the chosen genre again: all genres
      } else {
        state.genre = genre;
      }
      render();
    });
    genresBox.append(chip);
  }
}

// ---------- Start: draw the page for the first time ----------
render();
```

Step 10. My List ~25 min

Teaches: My List is an array of ids in the state: `push` adds, `filter` removes, `includes` checks.

TASK

1. In the `state` object, add `myList: []`.
2. At the top, find the counter in the header: `const listCount = document.querySelector("#list-count");`
3. Fill in **TODO 1** and **TODO 2** in `toggleMyList`.
4. In `createCard`, right before `return card;`, add the button: `card.append(createListButton(show));`
5. At the end of `render()`, show how many shows are in My List: `listCount.textContent = state.myList.length;`

Click + on a few cards and watch the counter at the top.

COMMON MISTAKES

- `state.myList.filter(...)` without `state.myList =` in front: `filter` makes a new array and doesn't change the old one.
- Adding the button after `return card;`, where the code never runs.
- `listCount.textContent` set outside `render()`, so it doesn't update on clicks.

A CORRECT ANSWER

```
// ... unchanged ...
const searchForm = document.querySelector("#search-form");
const emptyMessage = document.querySelector("#empty-message");
const listCount = document.querySelector("#list-count");

// ---------- One card for one show ----------
// ... unchanged ...
  body.append(badge, title, rating, meta);
  card.append(poster, body);
  card.append(createListButton(show));
  return card;
}
// ... unchanged ...
  search: "",   // the text in the search box
  genre: "",    // the chosen genre ("" means all genres)
  myList: [],   // the ids of the shows in My List, like [2, 5]
};

// ... unchanged ...
  renderGrid(list);
  renderGenres();
  listCount.textContent = state.myList.length;
}

// ... unchanged ...
// ---------- Start: draw the page for the first time ----------
render();

// ---------- My List ----------

// Is this show in My List? Gives true or false.
function isInMyList(id) {
  return state.myList.includes(id);
}
```

```
// Add the show to My List, or take it out if it's already there
function toggleMyList(show) {
  if (isInMyList(show.id)) {
    state.myList = state.myList.filter((id) => id !== show.id);
  } else {
    state.myList.push(show.id);
  }
  render();
}

// The round button on a card: "+" adds the show, "✓" means it's in My List
function createListButton(show) {
  const button = document.createElement("button");
  button.className = "list-btn";
  if (isInMyList(show.id)) {
    button.textContent = "✓";
  } else {
    button.textContent = "+";
  }
  button.setAttribute("aria-pressed", isInMyList(show.id));   // "pressed" look, and for screen
readers
  button.setAttribute("aria-label", "My List: " + show.name);
  button.addEventListener("click", () => {
    toggleMyList(show);
  });
  return button;
}
```

Step 11. Remember My List ~15 min

Teaches: `localStorage.setItem/getItem` with `JSON.stringify/parse` saves data across reloads.

TASK

1. Fill in the TODO in `saveMyList`, using the key `"myList"`.
2. In `toggleMyList`, call `saveMyList();` right before `render();`.
3. In `state`, start `myList` from the saved list: `JSON.parse(localStorage.getItem("myList")) ?? []`.

Add two shows, then press **Run again**: they're still in My List.

COMMON MISTAKES

- Saving the array without `JSON.stringify`: it's stored as `"2,5"`, which `JSON.parse` can't read back as an array.
- Forgetting `JSON.parse` when loading, so `myList` is a piece of text.
- Calling `saveMyList()` after `render()` in a different function, or not at all.

NOTE

If a student's page shows ticks from earlier experiments, they can click them away. The Lab keeps localStorage between reloads on purpose, like a real browser does.

A CORRECT ANSWER

```
// ... unchanged ...
  search: "",   // the text in the search box
  genre: "",    // the chosen genre ("" means all genres)
  myList: JSON.parse(localStorage.getItem("myList")) ?? [],   // saved ids, or an empty list
};

// ... unchanged ...
    state.myList.push(show.id);
  }
  saveMyList();
  render();
}
// ... unchanged ...
  return button;
}

// Save My List in the browser (localStorage keeps text only, so we turn it into JSON text)
function saveMyList() {
  localStorage.setItem("myList", JSON.stringify(state.myList));
}
```

Extra 1. Only My List ~15 min

Teaches: A checkbox's `checked` and `"change"` event can drive a filter in `render()`.

TASK

1. At the top: `const onlyListBox = document.querySelector("#only-list");`
2. Add `onlyMyList: false` to `state`.
3. Add a `"change"` listener to the checkbox that sets `state.onlyMyList` and calls `render()`.
4. In `render()`, before `renderGrid(list)`: if `state.onlyMyList` is true, keep only the shows that are in My List (`filter` with `isInMyList(show.id)`).

A CORRECT ANSWER

```
// ... unchanged ...
const emptyMessage = document.querySelector("#empty-message");
const listCount = document.querySelector("#list-count");
const onlyListBox = document.querySelector("#only-list");

// ---------- One card for one show ----------
// ... unchanged ...
  genre: "",    // the chosen genre ("" means all genres)
  myList: JSON.parse(localStorage.getItem("myList")) ?? [],   // saved ids, or an empty list
  onlyMyList: false,  // is "Only My List" ticked?
};

// ... unchanged ...
    list = getByGenre(list, state.genre);
  }
  if (state.onlyMyList) {
    list = list.filter((show) => isInMyList(show.id));
  }

  renderGrid(list);
// ... unchanged ...
  localStorage.setItem("myList", JSON.stringify(state.myList));
}

// Only My List: tick the box to see only your shows
onlyListBox.addEventListener("change", () => {
  state.onlyMyList = onlyListBox.checked;
  render();
});
```

Extra 2. Best rated first ~15 min

Teaches: `list.sort((a, b) => b.rating - a.rating)` sorts from highest to lowest.

TASK

1. At the top: `const sortSelect = document.querySelector("#sort");`
2. Add `sort: ""` to `state`.
3. Add a `"change"` listener to the select that sets `state.sort` to `sortSelect.value` and calls `render()`.
4. In `render()`, before `renderGrid(list)`: if `state.sort` is `"rating"`, sort `list` with the best rating first.

A CORRECT ANSWER

```
// ... unchanged ...
const emptyMessage = document.querySelector("#empty-message");
const listCount = document.querySelector("#list-count");
const sortSelect = document.querySelector("#sort");

// ---------- One card for one show ----------
// ... unchanged ...
  genre: "",    // the chosen genre ("" means all genres)
  myList: JSON.parse(localStorage.getItem("myList")) ?? [],   // saved ids, or an empty list
  sort: "",     // "" for normal order, "rating" for best rated first
};

// ... unchanged ...
    list = getByGenre(list, state.genre);
  }
  if (state.sort === "rating") {
    list.sort((a, b) => b.rating - a.rating);   // highest rating first
  }

  renderGrid(list);
// ... unchanged ...
  localStorage.setItem("myList", JSON.stringify(state.myList));
}

// The order drop-down
sortSelect.addEventListener("change", () => {
  state.sort = sortSelect.value;
  render();
});
```

Extra 3. A details box ~20 min

Teaches: `dialog.showModal()` opens a dialog; `event.stopPropagation()` stops a click bubbling up.

TASK

1. At the top, find the dialog: `const detailsDialog = document.querySelector("#details");`
2. Write `openDetails(show)`: set the text of `#details-title` (name), `#details-meta` (years) and `#details-summary` (`show.summary`), then call `detailsDialog.showModal()`.
3. In `createCard`, before `return card;`, add a click listener to the card that calls `openDetails(show)`.
4. The close button `#close-details` closes the dialog.
5. In `createListButton`, add `event.stopPropagation();` so the + button doesn't open the dialog.

A CORRECT ANSWER

```
// ... unchanged ...
const emptyMessage = document.querySelector("#empty-message");
const listCount = document.querySelector("#list-count");
const detailsDialog = document.querySelector("#details");

// ---------- One card for one show ----------
// ... unchanged ...
  card.append(poster, body);
  card.append(createListButton(show));
  card.addEventListener("click", () => {
    openDetails(show);
  });
  return card;
}
// ... unchanged ...
  button.setAttribute("aria-pressed", isInMyList(show.id));   // "pressed" look, and for screen
readers
  button.setAttribute("aria-label", "My List: " + show.name);
  button.addEventListener("click", (event) => {
    event.stopPropagation();   // don't also open the details box
    toggleMyList(show);
  });
// ... unchanged ...
  localStorage.setItem("myList", JSON.stringify(state.myList));
}

// ---------- Details box ----------
function openDetails(show) {
  document.querySelector("#details-title").textContent = show.name;
  document.querySelector("#details-meta").textContent = yearsLabel(show.premiered, show.ended);
  document.querySelector("#details-summary").textContent = show.summary;
  detailsDialog.showModal();
}

document.querySelector("#close-details").addEventListener("click", () => {
  detailsDialog.close();
});
```